TRACEFORGE V2 — Interactive Launcher

> **Component:** Interactive CLI Launcher
> **File:** `traceforge/launcher.py` **Run with:** `python -m traceforge.launcher` **Purpose:** ASCII banner with live case stats and quick-start menu — bridges the gap between professional CLI and approachable student interface

---

What this builds

A full-screen terminal launcher that shows:

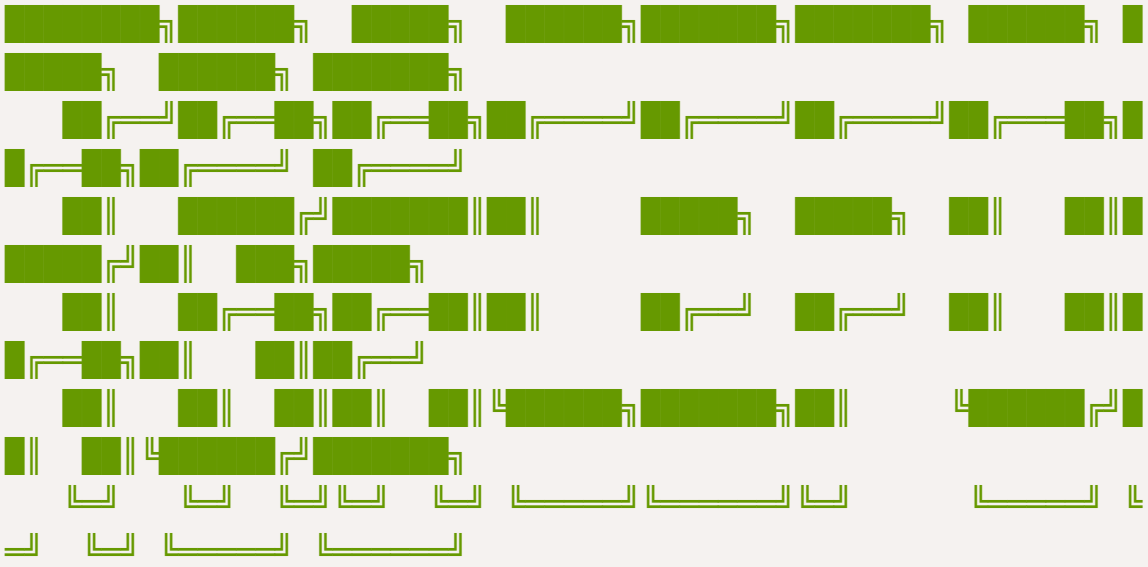
- The TraceForge ASCII banner
- Live stats pulled from the database — total cases, total artifacts, recent activity
- A numbered quick-start menu for common workflows
- Color-coded output using Rich

This is the first thing a user sees when they launch TraceForge. It does not replace the CLI subcommands — it wraps them in an approachable interface.

Step 1 — Create the launcher file

```
nano ~/traceforge/traceforge/launcher.py
```

```
"""
TraceForge v2 — Interactive Launcher
The main entry point for interactive use.
Run with: python -m traceforge.launcher
"""

import os
import sys
import subprocess
from pathlib import Path
```

```python
from datetime import datetime, timezone

from rich.console import Console
from rich.table import Table
from rich.panel import Panel
from rich.columns import Columns
from rich.text import Text
from rich import box
from loguru import logger

# Disable loguru output in launcher mode
logger.remove()

console = Console()

BANNER = """

"""

VERSION = "v2.0.0"
GITHUB  = "github.com/theconsoler/TRACEFORGE-V2"


def clear():
    os.system("clear")
```

```
def get_live_stats() -> dict:
    """Pull live stats from the database."""
    try:
        from traceforge.core.ledger import init_db, list_ca
ses
        from traceforge.core.store import get_artifact_coun
t
        init_db()
        cases = list_cases()
        total_artifacts = 0
        for c in cases:
            counts = get_artifact_count(c["id"])
            total_artifacts += sum(counts.values())
        return {
            "total_cases": len(cases),
            "total_artifacts": total_artifacts,
            "cases": cases[:5]
        }
    except Exception:
        return {"total_cases": 0, "total_artifacts": 0, "ca
ses": []}


def print_banner():
    """Print the TraceForge ASCII banner — cyan."""
    banner_text = Text(BANNER, style="bold cyan")
    console.print(banner_text)
    console.print(
        f"  [bold white]Digital Forensics & Incident Respon
se Toolkit[/bold white]  "
        f"[dim white]{VERSION}[/dim white]  [dim]{GITHUB}[/
dim]\n"
    )


def print_stats(stats: dict):
    """Print live case statistics — green panels."""
    panels = [
```

```
        Panel(
            f"[bold green]{stats['total_cases']}[/bold gree
n]\n[dim]Total Cases[/dim]",
            border_style="green",
            padding=(0, 2)
        ),
        Panel(
            f"[bold green]{stats['total_artifacts']}[/bold
green]\n[dim]Total Artifacts[/dim]",
            border_style="green",
            padding=(0, 2)
        ),
        Panel(
            f"[bold green]{datetime.now().strftime('%Y-%m-%
d')}[/bold green]\n[dim]Today[/dim]",
            border_style="green",
            padding=(0, 2)
        ),
    ]
    console.print(Columns(panels))
    console.print()


def print_recent_cases(stats: dict):
    """Print the most recent cases — yellow header."""
    if not stats["cases"]:
        console.print("  [dim]No cases found. Create your f
irst case with option 1.[/dim]\n")
        return

    table = Table(
        box=box.SIMPLE,
        show_header=True,
        header_style="bold yellow",
        padding=(0, 2)
    )
    table.add_column("Case ID", style="cyan", no_wrap=True)
    table.add_column("Name", style="white")
```

```
    table.add_column("Analyst", style="dim")
    table.add_column("Created", style="dim")

    for c in stats["cases"]:
        table.add_row(
            c["id"],
            c["name"],
            c["analyst"],
            c["created_at"][:10]
        )

    console.print("  [bold yellow]Recent Cases[/bold yello
w]")
    console.print(table)


def print_menu():
    """Print the quick-start menu — options in magenta, des
criptions in dim."""
    console.print("  [bold magenta]=== Main Menu ===[/bold
magenta]\n")
    menu_items = [
        ("1", "New Investigation Case",  "Create a new case
and begin evidence collection"),
        ("2", "Analyze Evidence",        "Run memory, dis
k, log, or network analysis"),
        ("3", "Correlate Findings",      "Run cross-module
correlation engine"),
        ("4", "Generate Report",         "Export case repo
rt as JSON, HTML, or PDF"),
        ("5", "View All Cases",          "List all investi
gation cases"),
        ("6", "Launch Web Dashboard",    "Open the Flask d
ashboard in browser"),
        ("7", "Run Full Pipeline",       "New case + analy
ze + correlate + report in one flow"),
        ("0", "Exit",                    "Quit TraceForg
e"),
```

```
    ]

    for num, title, desc in menu_items:
        if num == "0":
            console.print(
                f"  [bold red]{num}[/bold red]  [bold red]
{title:<30}[/bold red] [dim]{desc}[/dim]"
            )
        else:
            console.print(
                f"  [bold magenta]{num}[/bold magenta]  [bo
ld white]{title:<30}[/bold white] [dim]{desc}[/dim]"
            )
    console.print()


def handle_new_case():
    """Interactive new case creation."""
    console.print("\n[bold cyan]New Investigation Case[/bol
d cyan]\n")
    case_id  = console.input("  [cyan]Case ID[/cyan] (e.g.
CASE-2026-001): ").strip()
    name     = console.input("  [cyan]Case name[/cyan]: ").
strip()
    analyst  = console.input("  [cyan]Analyst name[/cyan]:
").strip()
    notes    = console.input("  [cyan]Notes[/cyan] (optiona
l): ").strip()

    if not case_id or not name or not analyst:
        console.print("[bold red]Case ID, name, and analyst
are required.[/bold red]")
        return

    subprocess.run([
        sys.executable, "-m", "traceforge.cli",
        "case", "new",
        "--id", case_id,
```

```
        "--name", name,
        "--analyst", analyst,
        "--notes", notes or ""
    ], capture_output=False)


def handle_analyze():
    """Interactive evidence analysis."""
    console.print("\n[bold cyan]Analyze Evidence[/bold cya
n]\n")
    case_id   = console.input("  [cyan]Case ID[/cyan]: ").s
trip()
    module    = console.input("  [cyan]Module[/cyan] [memor
y/disk/logs/network]: ").strip().lower()
    file_path = console.input("  [cyan]Evidence file path[/
cyan]: ").strip()
    analyst   = console.input("  [cyan]Analyst name[/cyan]:
").strip()
    host_id   = console.input("  [cyan]Host identifier[/cya
n] (optional): ").strip()

    if module not in ("memory", "disk", "logs", "network"):
        console.print("[bold red]Invalid module. Choose fro
m: memory, disk, logs, network[/bold red]")
        return

    cmd = [
        sys.executable, "-m", "traceforge.cli", "analyze",
        "--case", case_id,
        "--module", module,
        "--file", file_path,
        "--analyst", analyst,
    ]
    if host_id:
        cmd += ["--host", host_id]

    console.print(f"\n[bold green]Running {module.upper()}
analysis...[/bold green]")
```

```
    subprocess.run(cmd, capture_output=False)


def handle_correlate():
    """Interactive correlation."""
    console.print("\n[bold cyan]Correlation Engine[/bold cy
an]\n")
    case_id = console.input("  [cyan]Case ID[/cyan]: ").str
ip()
    window  = console.input("  [cyan]Time window in seconds
[/cyan] (default 30): ").strip()
    window  = window if window.isdigit() else "30"

    console.print(f"\n[bold green]Running correlation engin
e...[/bold green]")
    subprocess.run([
        sys.executable, "-m", "traceforge.cli", "correlat
e",
        "--case", case_id,
        "--window", window
    ], capture_output=False)


def handle_report():
    """Interactive report generation."""
    console.print("\n[bold cyan]Generate Report[/bold cyan]
\n")
    case_id = console.input("  [cyan]Case ID[/cyan]: ").str
ip()
    fmt     = console.input("  [cyan]Format[/cyan] [json/ht
ml/pdf/all] (default all): ").strip().lower()
    fmt     = fmt if fmt in ("json", "html", "pdf", "all")
else "all"

    console.print(f"\n[bold green]Generating {fmt.upper()}
report...[/bold green]")
    subprocess.run([
        sys.executable, "-m", "traceforge.cli", "report",
```

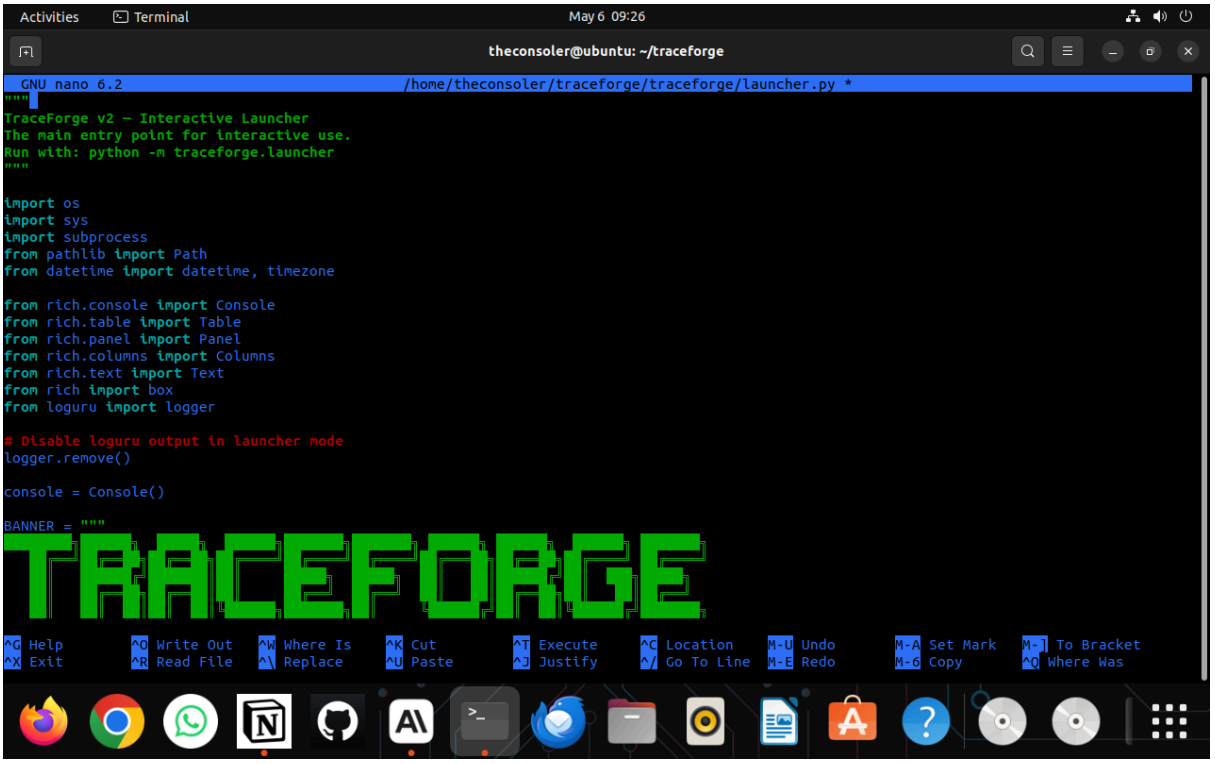
```
        "--case", case_id,
        "--format", fmt
    ], capture_output=False)


def handle_view_cases():
    """View all cases."""
    console.print("\n[bold cyan]All Cases[/bold cyan]\n")
    subprocess.run([
        sys.executable, "-m", "traceforge.cli", "case", "li
st"
    ], capture_output=False)


def handle_dashboard():
    """Launch the Flask dashboard."""
    console.print("\n[bold cyan]Launching Web Dashboard[/bo
ld cyan]")
    console.print("  Opening at: [bold green]http://127.0.
0.1:5000[/bold green]")
    console.print("  Press [bold red]Ctrl+C[/bold red] to s
top the dashboard\n")

    try:
        subprocess.run([
            sys.executable, "-m", "dashboard.app"
        ], capture_output=False)
    except KeyboardInterrupt:
        console.print("\n[yellow]Dashboard stopped.[/yello
w]")


def handle_full_pipeline():
    """Run the full pipeline interactively."""
    console.print("\n[bold cyan]Full Pipeline — New Case to
Report[/bold cyan]\n")
    console.print("  [dim]Create case → Analyze → Correlate
→ Report[/dim]\n")
```

```
    case_id = console.input("  [cyan]Case ID[/cyan]: ").str
ip()
    name    = console.input("  [cyan]Case name[/cyan]: ").s
trip()
    analyst = console.input("  [cyan]Analyst name[/cyan]:
").strip()

    console.print("\n[bold green]Step 1 — Creating case...
[/bold green]")
    subprocess.run([
        sys.executable, "-m", "traceforge.cli",
        "case", "new",
        "--id", case_id, "--name", name, "--analyst", analy
st
    ], capture_output=False)

    while True:
        console.print("\n[bold green]Step 2 — Add evidence
file[/bold green]")
        console.print("  [dim]Modules available: memory, di
sk, logs, network[/dim]")
        module = console.input("  [cyan]Module[/cyan] (or
'done' to skip to correlation): ").strip().lower()
        if module == "done":
            break
        if module not in ("memory", "disk", "logs", "networ
k"):
            console.print("[bold red]Invalid module.[/bold
red]")
            continue
        file_path = console.input("  [cyan]Evidence file pa
th[/cyan]: ").strip()
        host_id   = console.input("  [cyan]Host identifier
[/cyan] (optional): ").strip()

        cmd = [
            sys.executable, "-m", "traceforge.cli", "analyz
```

```
e",
            "--case", case_id, "--module", module,
            "--file", file_path, "--analyst", analyst
        ]
        if host_id:
            cmd += ["--host", host_id]
        subprocess.run(cmd, capture_output=False)

    console.print("\n[bold green]Step 3 — Running correlati
on engine...[/bold green]")
    subprocess.run([
        sys.executable, "-m", "traceforge.cli",
        "correlate", "--case", case_id
    ], capture_output=False)

    console.print("\n[bold green]Step 4 — Generating repor
t...[/bold green]")
    subprocess.run([
        sys.executable, "-m", "traceforge.cli",
        "report", "--case", case_id, "--format", "all"
    ], capture_output=False)

    console.print(f"\n[bold green]Pipeline complete for cas
e {case_id}[/bold green]")


def goodbye():
    """Print goodbye message on exit."""
    clear()
    console.print(Text(BANNER, style="bold cyan"))
    console.print("\n  [bold yellow]Thank you for using Tra
ceForge v2[/bold yellow]")
    console.print("  [dim]Stay sharp. Stay forensic.[/di
m]")
    console.print(f"\n  [dim]{GITHUB}[/dim]\n")


def main():
```

```
    """Main launcher loop."""
    try:
        while True:
            clear()
            print_banner()

            stats = get_live_stats()
            print_stats(stats)
            print_recent_cases(stats)
            console.print()
            print_menu()

            try:
                choice = console.input("  [bold magenta]Sel
ect an option:[/bold magenta] ").strip()
            except KeyboardInterrupt:
                goodbye()
                sys.exit(0)

            if choice == "1":
                handle_new_case()
            elif choice == "2":
                handle_analyze()
            elif choice == "3":
                handle_correlate()
            elif choice == "4":
                handle_report()
            elif choice == "5":
                handle_view_cases()
            elif choice == "6":
                handle_dashboard()
            elif choice == "7":
                handle_full_pipeline()
            elif choice == "0":
                goodbye()
                sys.exit(0)
            else:
                console.print("\n[bold red]Invalid option.
```

```
[/bold red] [dim]Please select a number from the menu.[/di
m]")

            try:
                console.input("\n  [dim]Press Enter to retu
rn to menu...[/dim]")
            except KeyboardInterrupt:
                goodbye()
                sys.exit(0)

    except KeyboardInterrupt:
        goodbye()
        sys.exit(0)


if __name__ == "__main__":
    main()
```

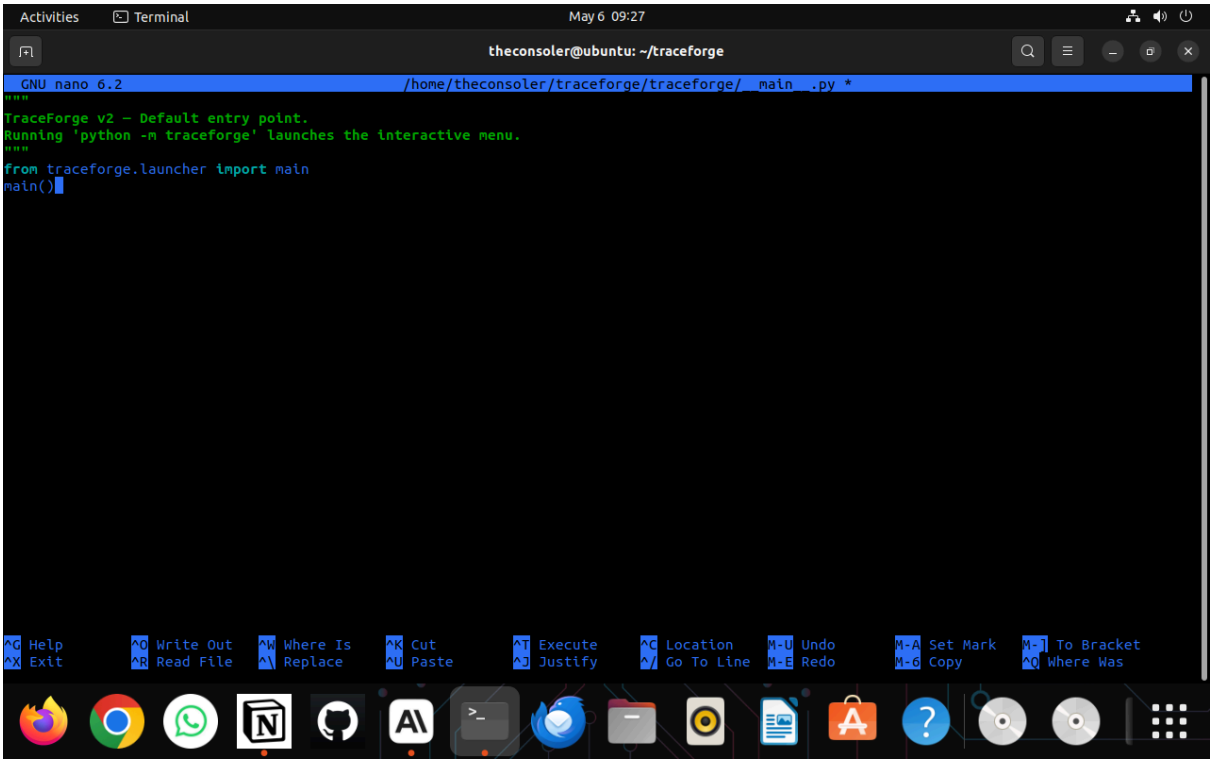


Step 2 — Add launcher entry to `__main__.py`

This lets you run the launcher with `python -m traceforge`:

```
nano ~/traceforge/traceforge/__main__.py
```

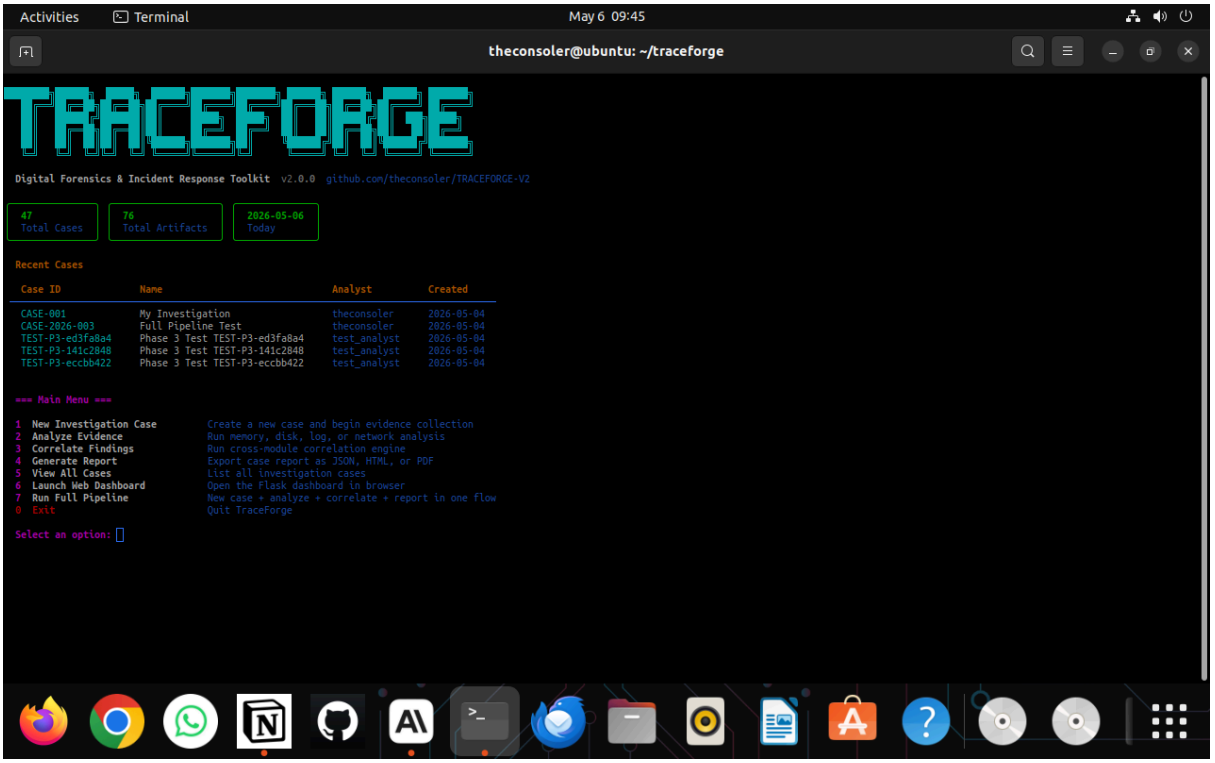
```
"""
TraceForge v2 — Default entry point.
Running 'python -m traceforge' launches the interactive men
u.
"""
from traceforge.launcher import main
main()
```



Step 3 — Run the launcher

```
cd ~/traceforge
source venv/bin/activate
python -m traceforge
```

You should see the full ASCII banner, live stats from your database, recent cases listed, and the numbered menu.



What each menu option does

Option	What it does
1	Prompts for case ID, name, analyst — creates the case
2	Prompts for case ID, module, file path — runs analysis and stores artifacts
3	Prompts for case ID — runs correlation engine
4	Prompts for case ID and format — generates report
5	Lists all cases in a table
6	Starts the Flask dashboard at port 5000
7	Full guided pipeline — case creation through report in one flow
0	Exit

Step 4 — Commit and push

```
cd ~/traceforge
git add .
git commit -m "feat: interactive launcher with ASCII banne
```

```
r, live stats, and quick-start menu"
git push origin main
```

What this looks like when running

```
████████╗██████╗  ████╗  ██████╗███████╗...
  Digital Forensics & Incident Response Toolkit  v2.0.0

  ┌──────────────┐ ┌──────────────────┐ ┌──────────────┐
  │      3       │ │       17         │ │  2026-05-05  │
  │ Total Cases  │ │ Total Artifacts  │ │    Today     │
  └──────────────┘ └──────────────────┘ └──────────────┘

  Recent Cases
  CASE-001   My Investigation   theconsoler   2026-05-04
  CASE-002   Phase 2 Test       theconsoler   2026-05-04

  === Main Menu ===

  1  New Investigation Case        Create a new case
  2  Analyze Evidence              Run analysis modules
  3  Correlate Findings            Cross-module correlation
  4  Generate Report               Export JSON, HTML, PDF
  5  View All Cases                List all cases
  6  Launch Web Dashboard          Open Flask dashboard
  7  Run Full Pipeline             End-to-end workflow
  0  Exit

  Select an option:
```

Will the launcher work with the dashboard?

Yes, completely. Here is why:

The launcher, the CLI, and the dashboard all read and write to the **same SQLite database** at `~/traceforge/data/traceforge.db`. They are three different interfaces to the same data. It does not matter which one you use to create a case or run analysis — the data ends up in the same place.

So the flow works like this:

```
Launcher (menu option 2)
    → calls CLI analyze command
    → module runs and saves artifacts to SQLite
```

```
    → dashboard reads same SQLite
    → case appears in dashboard automatically
```

Nothing extra needs to be done. You create a case in the launcher, open the dashboard, and the case is already there.

How cases show in the dashboard

Step 1 — Create case and analyze via launcher:

```
python -m traceforge    ← opens launcher
Select option 1         ← create case
Select option 2         ← analyze evidence
Select option 3         ← correlate
```

Step 2 — Open dashboard in a second terminal:

```
source venv/bin/activate
python -m dashboard.app
```

Step 3 — Open browser:

```
http://127.0.0.1:5000
```

The home page shows every case as a row in a table with artifact counts and module badges. Click any case to see the full overview, timeline, correlations, and export buttons.

The exact data flow

```
Launcher menu
     ↓
CLI subcommand runs
     ↓
Module analyzes evidence file
     ↓
SHA-256 hash written to evidence ledger
     ↓
Artifacts saved to artifacts table
     ↓
     ┌──────────────────────────┐
```

```
        |   traceforge.db          |
        |   cases table            |
        |   evidence_log table     |
        |   artifacts table        |
        └──────────────────────────┘

              ↑                ↑
      CLI reads        Dashboard reads
```

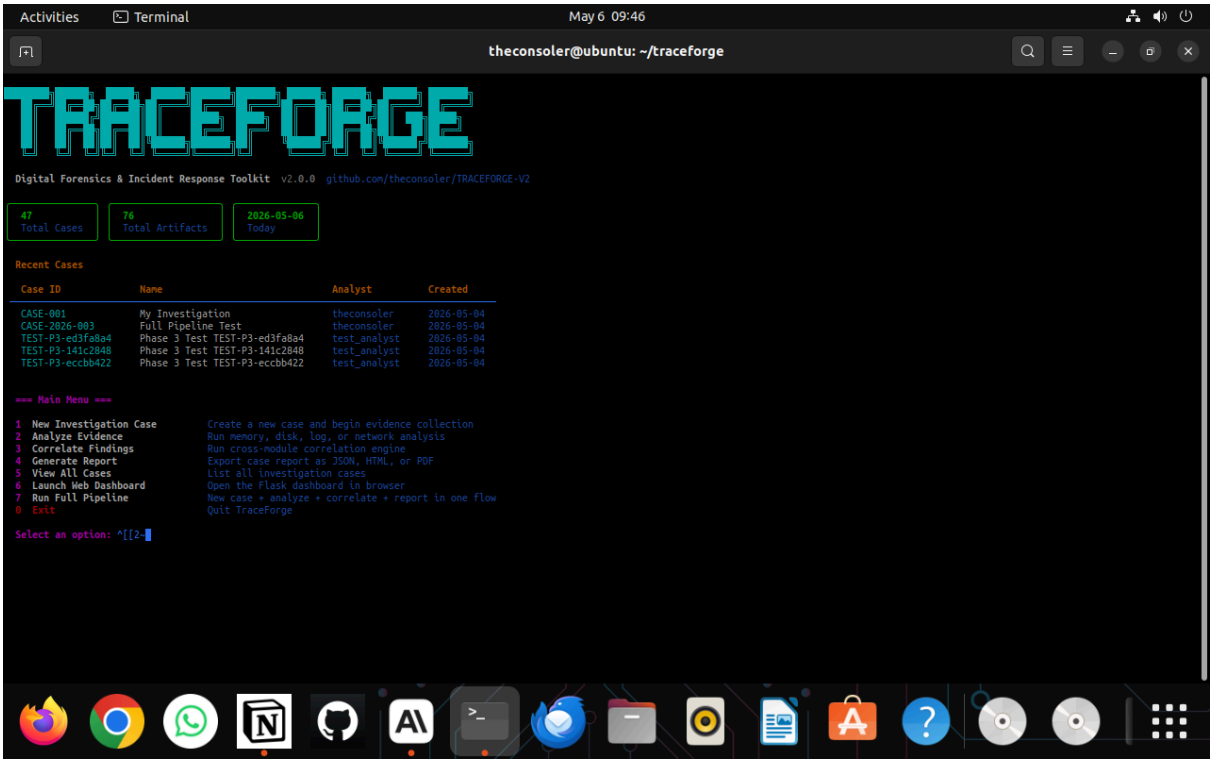
Both the CLI and dashboard point at the same file. There is no sync needed, no API calls between them, no data duplication. One writes, the other reads, instantly.

One thing to know

You cannot run the launcher and the dashboard at the same time in the same terminal window. Use two terminals:

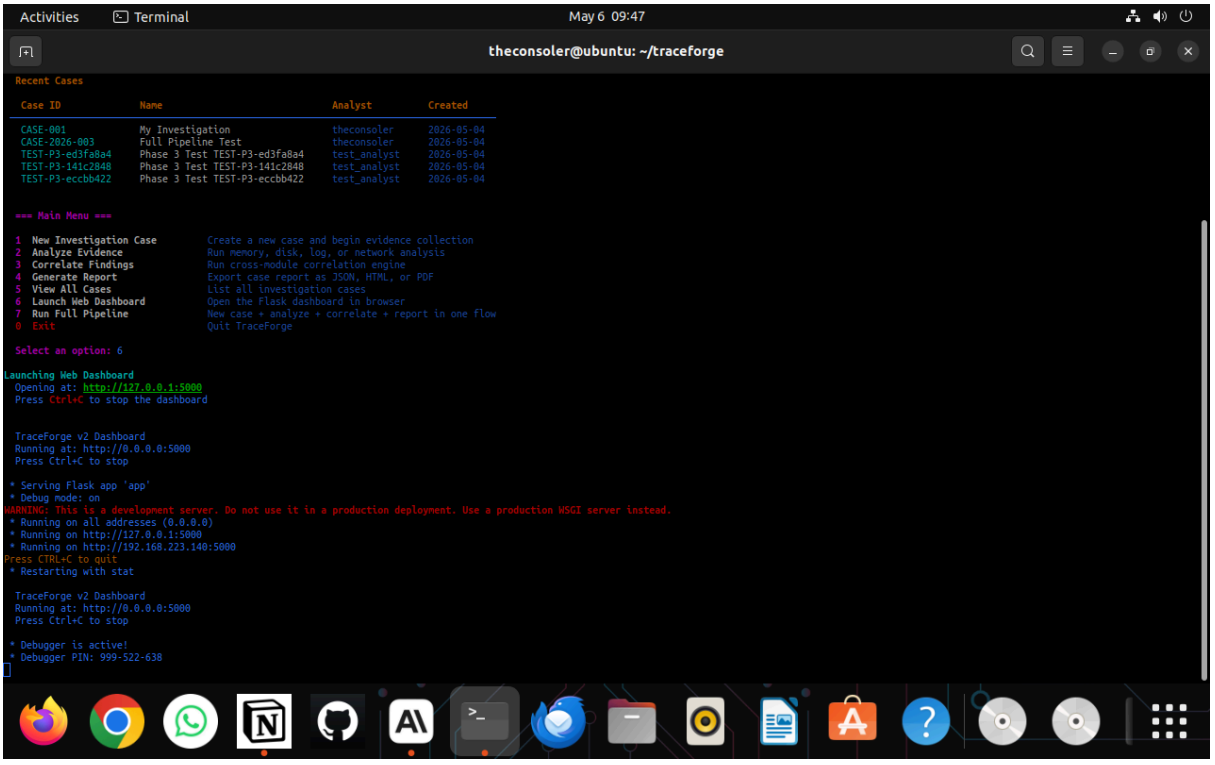
Terminal 1:

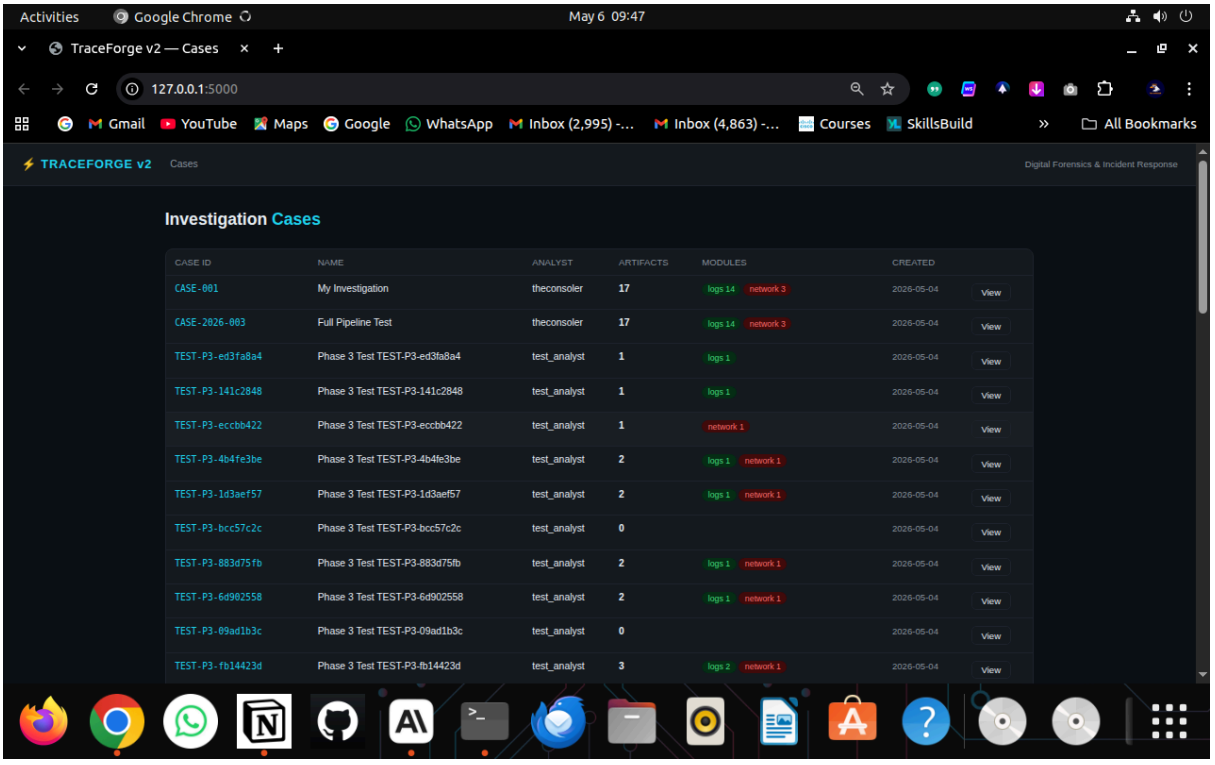
```
python -m traceforge    ← launcher for running investigatio
ns
```



Terminal 2:

```
python -m dashboard.app ← dashboard for viewing results
```





and its working perfectly.

That is the intended workflow. Investigate via launcher, view and present via dashboard.

TraceForge v2 — Interactive LauncherLast updated: May 2026